Unity 2D Space Shooter - Setup and Build Instructions

This project contains complete production-ready C# scripts in:

- `Assets/Scripts/GameManager.cs`
- `Assets/Scripts/PlayerController.cs`
- `Assets/Scripts/EnemyController.cs`
- `Assets/Scripts/BulletController.cs`
- `Assets/Scripts/EnemySpawner.cs`
- `Assets/Scripts/UIManager.cs`
- `Assets/Scripts/MainMenuController.cs`

---

1) Set up the project in Unity

1. Open **Unity Hub**.
2. Click **New project** → choose **2D (Core)** template.
3. Set project name (for example `SpaceShooter`) and location as desired.
4. Create the project.
5. Copy the `Assets/Scripts` folder from this repository into your Unity project `Assets/` folder.
6. In Unity, wait for script compilation to finish.

---

2) Create GameObjects and attach scripts

You need two scenes:
- `MainMenu`
- `GameScene`

Save both under `Assets/Scenes/`.

A. MainMenu scene setup

1. Create a new scene and save as `MainMenu`.
2. Create `Canvas` (Unity auto-creates `EventSystem`).
3. Add UI:
   - Title text (e.g., `SPACE SHOOTER`)
   - `Start Game` button
   - `Quit` button
4. Create empty GameObject named `MainMenuManager`.
5. Attach `MainMenuController` script to `MainMenuManager`.
6. On `Start Game` button OnClick, drag `MainMenuManager` and select:
   - `MainMenuController.OnStartGameButtonPressed()`

7. On `Quit` button OnClick, drag `MainMenuManager` and select:
   - `MainMenuController.OnQuitButtonPressed()`

B. GameScene setup

1. Create a new scene and save as `GameScene`.
2. Ensure camera is orthographic (default in 2D).

Core Managers

1. Create empty GameObject `GameManager` and attach `GameManager` script.
2. Create empty GameObject `UIManager` and attach `UIManager` script.

Player Setup

1. Create player object:
   - `GameObject -> 2D Object -> Sprite` (rename to `Player`)
   - Add `Rigidbody2D`:
     - Body Type: Dynamic
     - Gravity Scale: 0
     - Freeze Rotation Z = true
     - Add `Collider2D` (BoxCollider2D or CircleCollider2D)
     - Enable **Is Trigger**
     - Attach `PlayerController`
2. Create child object under Player named `FirePoint` and place slightly above the ship.
3. Assign `FirePoint` to PlayerController.

Bullet Prefab Setup

1. Create bullet sprite object named `Bullet`:
   - Add `Collider2D` (Is Trigger = true)
   - Add `Rigidbody2D`:
     - Body Type: Kinematic
     - Gravity Scale: 0
     - Attach `BulletController`
2. Drag bullet object into `Assets/Prefabs/` to create prefab `Bullet.prefab`.
3. Delete bullet from scene after prefab creation.
4. Assign `Bullet.prefab` to:
   - `PlayerController.bulletPrefab`
   - `EnemyController.bulletPrefab` (on enemy prefab)

Enemy Prefab Setup

1. Create enemy sprite object named `Enemy`:
   - Add `Collider2D` (Is Trigger = true)
   - Add `Rigidbody2D`:
     - Body Type: Kinematic
     - Gravity Scale: 0
     - Attach `EnemyController`
2. Create child `FirePoint` under enemy (slightly below ship front) and assign in `EnemyController` if using enemy shooting.
3. Configure enemy shooting:
   - `canShoot = true` (optional)
   - `shootInterval` as desired

4. Create prefab by dragging enemy object into `Assets/Prefabs/Enemy.prefab`.
5. Delete scene enemy instance.

Enemy Spawner

1. Create empty GameObject `EnemySpawner`.
2. Attach `EnemySpawner` script.
3. In inspector:
   - Add element to `enemyPrefabs` and assign `Enemy.prefab`
   - Tune spawn range (`minX`, `maxX`) and `spawnY` to match camera bounds.

UI Setup (HUD + Game Over)

1. In Canvas, create:
   - `ScoreText` (top-left)
   - `HealthText` (top-right)
2. Create `GameOverPanel` (disabled by default) containing:
   - `Game Over` text
   - `FinalScoreText`
   - `Restart` button
   - `Main Menu` button
3. Assign references in `UIManager`:
   - `scoreText` -> ScoreText
   - `healthText` -> HealthText
   - `gameOverPanel` -> GameOverPanel
   - `finalScoreText` -> FinalScoreText
4. Wire GameOver buttons:
   - Restart button OnClick -> `UIManager.OnRestartButtonPressed()`
   - Main Menu button OnClick -> `UIManager.OnMainMenuButtonPressed()`

Player Script References

1. In PlayerController inspector:
   - Assign `bulletPrefab` (`Bullet.prefab`)
   - Assign `firePoint` (Player/FirePoint)
   - Tune movement/combat values as desired.

---

3) Physics and collision configuration

Recommended tags (optional but helpful for organization): `Player`, `Enemy`, `Bullet`.

Important component rules:
- Player, Enemy, and Bullet colliders should be **Is Trigger = true**.
- Use Rigidbody2D on moving objects to ensure trigger callbacks fire.
- Gravity scale should be `0` for all ships and bullets.

Collision behavior implemented in scripts:
- Player bullets damage enemies.
- Enemy bullets damage player.
- Player touching enemy causes player damage and destroys enemy.
- Enemy death increases score.
- Player health reaching 0 triggers game over.

4) Build settings for Windows and build executable

1. Open **File -> Build Settings**.
2. Click **Add Open Scenes** for:
   - `MainMenu` (set as Scene index 0)
   - `GameScene` (Scene index 1)
3. Select platform: **Windows, Mac, Linux Standalone**.
4. Target Platform: **Windows**.
5. Architecture: **x86_64**.
6. Click **Switch Platform** (if needed).
7. Click **Player Settings** and set:
   - Company/Product name
   - Resolution defaults (optional)
   - Fullscreen mode preference
8. Click **Build**.
9. Choose output folder (e.g., `Builds/Windows/`).
10. Unity generates `.exe` + data folder.
11. Run the generated `.exe` to play.

Gameplay controls

- Move: **WASD** or **Arrow Keys**
- Shoot: **Space bar**

Notes

- This is a complete script set for a functional 2D space shooter.
- You can extend difficulty by adding multiple enemy prefabs with different speed/health/fire rates and assigning all of them in `EnemySpawner.enemyPrefabs`.